



UNIVERSIDADE ESTADUAL DE SANTA CRUZ

Larissa de Brito Santos

**RELATÓRIO 1: IMPLEMENTAÇÃO PARALELA COM
OPENMP**

**Ilhéus – BA
2026**

Larissa de Brito Santos

RELATÓRIO 1: IMPLEMENTAÇÃO PARALELA COM OPENMP

Relatório técnico apresentado como requisito parcial para disciplina DEC107 — Processamento Paralelo, referente à etapa 1 do projeto da matéria: Benchmark de Geração do Conjunto de Mandelbrot com OpenMP.

Professor: Esbel Tomas Valero Orellana

SUMÁRIO

1	INTRODUÇÃO	4
2	METODOLOGIA	6
2.1	Especificações e Parâmetros do Benchmark	6
2.2	Implementação Sequencial de Referência	7
2.3	Implementação Paralela com OpenMP	9
2.4	Casos de Testes	10
2.4.1	Automação dos Experimentos	11
2.5	Ambiente de Execução e Hardware	12
2.6	Procedimentos de Medição de Tempo e Métricas de Avaliação	12
3	RESULTADOS	14
3.1	Validação de Corretude	14
3.2	Desempenho no Cenário Padrão	14
3.3	Políticas de Escalonamento e Efeito da Granularidade	16
3.3.1	Evolução do Escalonamento sob Altas Resoluções (8K e 16K)	17
3.3.2	Comportamento Longitudinal das Políticas por Cenário	17
3.4	Cenário de Desbalanceamento Acentuado (Vale dos Cavalos-Marinheiros)	19
3.5	Efeito do Paralelismo 2D (collapse) e Escalabilidade por Resolução	20
4	DISCUSSÃO	23
4.1	Análise da Corretude da Implementação	23
4.2	Comparação entre Abordagens Paralelas e Relação com a Execução Sequencial	24
4.3	Gargalos Encontrados e o Custo de Entrada e Saída (I/O)	25
4.4	Sugestões de Otimizações Futuras	25
5	CONCLUSÃO	26
	REFERÊNCIAS	28
	ANEXO A—ENUNCIADO DO BENCHMARK DE MANDELBROT	29

1 INTRODUÇÃO

Este trabalho tem como objetivo desenvolver e avaliar o desempenho de duas versões para o cálculo do Conjunto de Mandelbrot: uma sequencial e outra paralela, utilizando a biblioteca OpenMP para a segunda implementação. O foco principal desta Etapa 1 é medir os ganhos de tempo obtidos ao dividir o trabalho entre múltiplos núcleos de processamento, observando como diferentes formas de distribuir as tarefas afetam o resultado final.

O Conjunto de Mandelbrot é formado por números no plano complexo calculados a partir da fórmula $z_{n+1} = z_n^2 + c$, começando com $z_0 = 0$. Para saber se um ponto pertence ou não ao conjunto, foi usado o método de tempo de escape (escape-time): a fórmula é repetida até que o valor de $|z_n|$ ultrapasse o limite de corte (definido como 2 para esse projeto) ou até que se atinja um limite máximo de iterações (*MAX_ITER*, definido como 1000 para casos base nesse projeto). Indica-se o material (Mathigon, 2026) para um melhor entendimento da história e da matemática por trás do Conjunto de Mandelbrot.

Como todos os núcleos do computador acessam a mesma memória principal, o OpenMP facilita a criação de threads para calcular vários pontos ao mesmo tempo. Para controlar como as linhas ou pontos da imagem são distribuídos entre essas threads, o OpenMP oferece três políticas principais de escalonamento:

- **Static:** divide as iterações em blocos de tamanho fixo antes do laço começar e distribui de forma predeterminada entre as threads;
- **Dynamic:** entrega blocos de trabalho para as threads sob demanda, conforme cada uma termina a tarefa anterior e solicita mais;
- **Guided:** distribui o trabalho sob demanda de forma parecida com a *Dynamic*, mas começa entregando blocos grandes e vai reduzindo o tamanho das fatias ao longo da execução para diminuir a sobrecarga de comunicação.

Para entender o impacto de cada uma dessas abordagens no tempo total de execução, foram realizados testes práticos utilizando as três políticas de escalonamento. Cada uma foi avaliada com diferentes configurações de tamanho de bloco (*chunk size*) e variando a quantidade de *threads*, permitindo observar na prática qual combinação lida melhor com o desbalanceamento característico do cálculo.

Esse desbalanceamento acontece pois um ponto fora da figura é descartado em apenas duas ou três iterações, consumindo pouco processamento. Já um ponto dentro da figura ou muito próximo de sua borda precisa rodar todas as iterações até *MAX_ITER*.

Como as regiões pesadas ficam concentradas em partes específicas da imagem, uma divisão simples e fixa das linhas pode sobrecarregar uma thread enquanto outras terminam rápido e ficam sem fazer nada. Por essa razão, entender e medir o balanceamento de carga é o ponto central deste estudo.

Para comparar o desempenho das diferentes abordagens em memória compartilhada, este trabalho utiliza quatro métricas: o Tempo de Execução, que mede a duração total do cálculo em segundos; o Speedup, que expressa o ganho de velocidade da versão paralela em relação à sequencial; a Eficiência, que indica o aproveitamento real de cada núcleo de processamento utilizado; e o Fator de Balanceamento de Carga, empregado para quantificar se o trabalho foi dividido por igual entre as threads ou se algumas ficaram esperando as outras terminarem.

2 METODOLOGIA

2.1 Especificações e Parâmetros do Benchmark

As regras e os parâmetros adotados nesta etapa foram definidos com base no Anexo A, disponibilizado a toda a turma para garantir a padronização dos experimentos e a comparação justa entre os resultados.

O cálculo segue o método de tempo de escape (*escape-time*): para cada pixel (p_x, p_y) , obtém-se o número complexo c correspondente por mapeamento linear e repete-se a fórmula $z_{n+1} = z_n^2 + c$, partindo de $z_0 = 0$. O laço termina quando $|z|^2 > 4$ (equivalente a $|z| > 2$, evitando o custo de calcular raízes quadradas) ou quando se atinge o limite MAX_ITER . Todas as contas utilizam números de ponto flutuante em precisão dupla (*double*).

Os parâmetros oficiais e cenários de teste dividem-se em:

- **Cenário Padrão (Caso Base):**

- Região no plano complexo: $Re \in [-2,0; 1,0]$ e $Im \in [-1,5; 1,5]$;
- Resolução: 4096×4096 pixels;
- Limite de iterações: $MAX_ITER = 1000$;
- Critério de escape: $|z|^2 > 4$.

- **Casos de Escalabilidade e Estresse:**

- *Escalabilidade Forte (Strong Scaling)*: mantém a resolução fixa em 4096×4096 e varia a quantidade de *threads*;
- *Escalabilidade Fraca (Weak Scaling)*: aumenta proporcionalmente a resolução da imagem (por exemplo, 4096^2 , 8192^2 e 16384^2) conforme mais recursos de processamento são adicionados;
- *Desbalanceamento Acentuado*: região de aproximação (*zoom*) no “Vale dos Cavalos-Marinheiros”, com centro em $(-0,743643887; 0,131825904)$, largura de $3,0 \times 10^{-3}$ no eixo real (mantendo a proporção da tela) e $MAX_ITER = 5000$. Esse caso reúne uma grande quantidade de pixels de custo máximo para testar o comportamento das políticas de escalonamento.

A saída oficial é uma matriz com a contagem de iterações gravada em arquivo binário (inteiros de 32 bits, em ordem *row-major*), acompanhada de uma imagem PGM

ou PPM para verificação visual. O teste de corretude consiste na comparação direta dessa matriz binária da versão paralela com a versão sequencial de referência, admitindo-se divergências em no máximo 0,01% dos pixels de borda (com diferença máxima de ± 1 iteração), decorrentes de pequenas variações de arredondamento em ponto flutuante.

2.2 Implementação Sequencial de Referência

Para o desenvolvimento do projeto, a linguagem C++ foi escolhida por facilitar a organização dos dados e do fluxo do programa. Foi criada uma classe própria para representar os números complexos com as operações matemáticas necessárias definidas, além do aproveitamento das facilidades nativas da linguagem para salvar os arquivos de saída e imprimir informações no terminal.

Inicialmente, foi desenvolvida uma versão sequencial chamada `sequencial.cpp`. De acordo com a definição documentada por Wikipedia Contributors (2026), implementou-se uma otimização sugerida para a figura clássica do Mandelbrot: calcular apenas a metade superior da matriz e espelhar os valores verticalmente no eixo horizontal, aproveitando a simetria da imagem completa. Apesar de alguns desafios iniciais para acertar a lógica do espelhamento, a técnica reduziu muito o esforço computacional e entregou tempos excelentes. Contudo, essa abordagem foi descartada como referência oficial porque falhou no caso de teste do Vale dos Cavalos-Marinheiros. Como o programa foi projetado para calcular qualquer intervalo do plano complexo, regiões de *zoom* assimétricas não admitem espelhamento e geram resultados incorretos.

Por conta dessa limitação, desenvolveu-se a versão definitiva de referência, chamada `ponto_a_ponto.cpp`. Esse código calcula todos os pontos da matriz, um por um, sem atalhos geométricos. Durante os primeiros testes dessa versão, um erro na ordem dos índices da matriz gerou a imagem de cabeça para baixo. Corrigida a indexação das coordenadas, o programa produziu a saída correta para qualquer intervalo configurado, tornando-se a base sólida de validação de corretude para as versões paralelas. Essa validação foi confirmada pela comparação exata, elemento a elemento, entre a matriz gerada pelo código desenvolvido e a matriz de referência obtida pelo *script* em Python disponibilizado para a turma.

Com a matriz calculada, a etapa seguinte consistiu na coloração da imagem para inspeção visual. No arquivo PPM gerado, o código implementa uma suavização estruturada em quatro etapas matemáticas. Inicialmente, obtém-se a contagem de iterações através do algoritmo de *escape-time*. Caso o ponto atinja o limite máximo ($temp \geq MAX_ITER$), é atribuída a ele a cor preta (0,0,0), identificando-o como pertencente ao conjunto. Para os pontos que divergem, o valor de escape é normalizado no intervalo $[0, 1]$ pela razão $t_{base} = \frac{temp}{MAX_ITER}$ e submetido a uma transformação via raiz cúbica ($t = \sqrt[3]{t_{base}}$). Essa

correção comprime a amplitude das iterações, compensando a distribuição assimétrica do plano e realçando os filamentos finos e detalhes externos. Por fim, existe uma função que interpola os canais RGB com polinômios de Bernstein de grau 4:

$$R(t) = 9 \cdot (1 - t) \cdot t^3 \times 255$$

$$G(t) = 15 \cdot (1 - t)^2 \cdot t^2 \times 255$$

$$B(t) = 8,5 \cdot (1 - t)^3 \cdot t \times 255$$

Essa parametrização assegura que o canal azul atinja o pico em iterações baixas ($t \approx 0,25$), o verde em iterações intermediárias ($t = 0,50$) e o vermelho próximo à fronteira ($t \approx 0,75$). Dessa forma, o mapeamento produz uma transição cromática contínua que se inicia em tons de azul escuro nas áreas de escape rápido, avança por ciano e tons alaranjados, e escurece progressivamente até o preto junto à borda do fractal, conforme ilustrado na Figura 1.

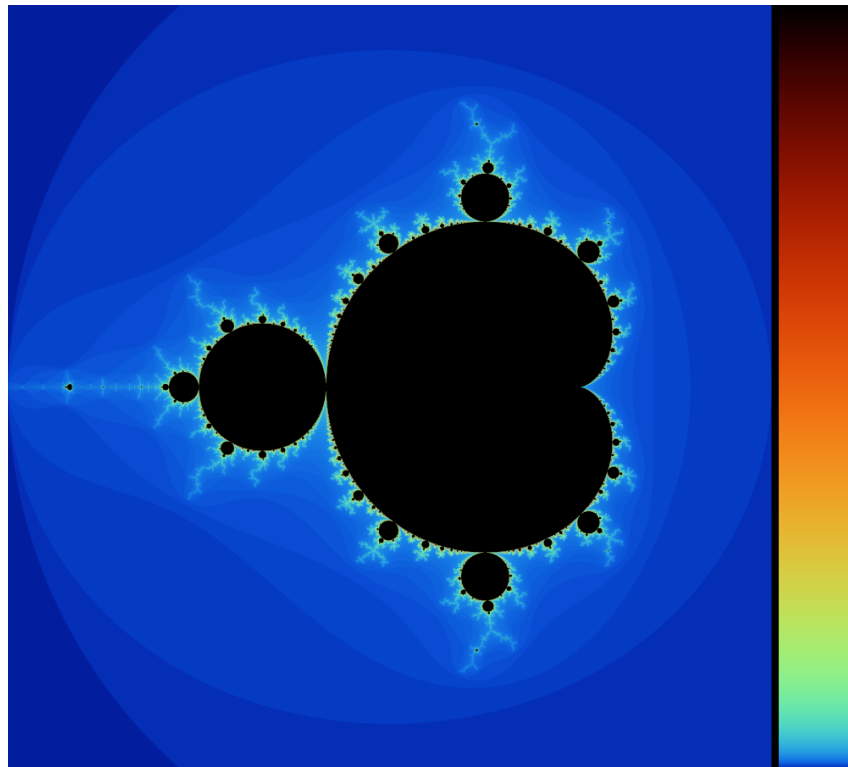
Figura 1 – Escala cromática contínua calculada através de polinômios de Bernstein.



Fonte: Própria.

Para facilitar a interpretação dos resultados visuais, o código acopla automaticamente uma barra lateral com a escala de cores à direita do fractal, separada por uma linha preta. Essa barra projeta verticalmente o gradiente completo de iterações, servindo como uma legenda integrada que correlaciona diretamente cada região da figura à sua respectiva faixa de custo computacional (Figura 2).

Figura 2 – Exemplo de renderização do Conjunto de Mandelbrot com barra lateral de cores.



Fonte: Própria.

2.3 Implementação Paralela com OpenMP

A paralelização do algoritmo foi realizada em C++ com o uso da biblioteca OpenMP (OpenMP Architecture Review Board, 2015), concentrando a otimização no cálculo do tempo de escape e preservando as etapas de leitura e escrita em disco de modo sequencial. Para este estudo, foram desenvolvidas duas implementações distintas: uma primeira versão (`paralelo.cpp`), que particiona o trabalho em nível de linhas, e, posteriormente, uma evolução direta (`paralelo_collapse.cpp`), voltada a avaliar o impacto de uma granularidade mais fina de divisão de tarefas, que particiona o trabalho em pixels.

Na implementação base (`paralelo.cpp`), a paralelização foi estruturada pela diretiva `#pragma omp parallel for` aplicada exclusivamente ao laço externo, correspondente às linhas da imagem. Essa escolha inicial buscou diminuir o *overhead* de gerenciamento do conjunto de *threads*, instanciando-o uma única vez para processar todo o quadro, evitando o custo de sincronização que ocorreria a cada linha caso a diretiva fosse inserida no laço interno. Para permitir a avaliação de desempenho sem a necessidade de recompilar o binário a cada teste, utilizou-se a cláusula `schedule(runtime)` associada à função `omp_set_schedule()`, tornando viável configurar a política de escalonamento (`static`,

`dynamic` ou `guided`) e a granularidade via parâmetros de entrada no arquivo `in.txt`.

Como evolução dessa abordagem, elaborou-se o código `paralelo_collapse.cpp`. Nessa segunda versão, foi incorporada a cláusula `collapse(2)` à diretiva paralela, fundindo os dois laços aninhados (linhas e colunas) em uma única sequência de iterações. Para assegurar a fusão, os laços foram construídos de forma perfeitamente aninhada, sem instruções intermediárias, deslocando toda a declaração de variáveis locais para o interior do laço interno.

Essa mudança estrutural redefine a interpretação prática do tamanho de bloco (*chunk size*) sob a cláusula `schedule(runtime)`. Enquanto na versão base um bloco de valor 64 delegava 64 linhas inteiras para uma *thread*, na versão com `collapse(2)` o mesmo valor passa a indicar apenas 64 *pixels*. Essa divisão no nível de pixel pode ser vantajosa em regiões de extremo desbalanceamento de carga, como no Vale dos Cavalos-Marinheiros, pois permite que núcleos distintos calculem trechos alternados de uma mesma linha pesada simultaneamente, em vez de reter uma única *thread* sobrecarregada com o cálculo daquela faixa. Isso será avaliado nos testes.

Apesar das diferenças na distribuição das iterações, ambas as versões compartilham o mesmo modelo de gestão de memória e sincronização. O vetor unidimensional de saída é compartilhado entre as *threads* por meio da cláusula `shared(temps)`. Não há concorrência de escrita: na versão por linhas, cada *thread* acessa faixas exclusivas da matriz, e na versão com *collapse*, o mapeamento associa cada coordenada a uma posição única na memória, tornando desnecessário o uso de travas como `atomic` ou `critical`. As variáveis auxiliares de cálculo foram declaradas dentro do laço, ficando privadas na pilha de cada *thread* sem uso explícito da cláusula `private`. Além disso, a diretiva `nowait` não foi utilizada para preservar a barreira de sincronização ao final do laço, o que garante que todos os pontos sejam calculados antes da gravação do arquivo. Também não foram utilizadas cláusulas de redução pela inexistência de acumuladores globais na versão sequencial do problema.

2.4 Casos de Testes

Para investigar o impacto da distribuição de trabalho sobre o desempenho e o balanceamento de carga, foram avaliadas três políticas de escalonamento fundamentais da biblioteca OpenMP (`static`, `dynamic` e `guided`), além da política de referência (`auto`). Em cada abordagem, o parâmetro de granularidade (*chunk size*) foi variado estrategicamente entre valores maiores e menores para avaliar os diversos cenários.

- **Escalonamento Estático (`static`):** O trabalho é dividido em partes iguais e distribuído para as *threads* antes do início do laço, com quase nenhum custo extra de organização. O bloco de tamanho 1024 foi usado para testar o caso de menor

custo de divisão, mas com maior risco de desbalancear a carga, enquanto o bloco de tamanho 64 foi testado para tentar espalhar melhor o trabalho entre os núcleos.

- **Escalonamento Dinâmico (dynamic):** Usa uma fila central em que cada *thread* pega um novo pedaço de trabalho assim que termina o anterior:
 - `dynamic:1`: Entrega a menor fatia possível, garantindo divisão de carga perfeita, mas com alto custo de espera por acessar a fila a todo momento.
 - `dynamic:1024`: Entrega fatias bem maiores, diminuindo os pedidos à fila e equilibrando melhor a divisão com a velocidade.
- **Escalonamento Guiado (guided):** As *threads* começam pegando blocos grandes de trabalho e, com o tempo, esses blocos vão diminuindo automaticamente. Os tamanhos 64 e 1024 definem o menor tamanho de fatia permitido na fase final, sendo que 64 ajuda mais a equilibrar o final do processamento do que 1024.
- **Escalonamento Automático (auto):** Deixa o próprio OpenMP escolher a melhor estratégia para o laço, servindo como ponto de comparação para ver se as escolhas manuais obtêm melhor resultado.

2.4.1 Automação dos Experimentos

A execução sistemática dos testes foi orquestrada pelo script `run_tests.py`, responsável por padronizar as condições experimentais. O fluxo experimental abrangeu as seguintes etapas:

- **Compilação Otimizada:** Os códigos sequencial (`ponto_a_ponto.cpp`) e paralelos (`paralelo.cpp` e `paralelo_collapse.cpp`) foram compilados via `g++` empregando as diretivas de alto desempenho `-O3` e `-ffast-math`.
- **Exploração Combinatória de Parâmetros:** Para cada programa, o script gerou o produto cartesiano envolvendo as resoluções de grade (4096×4096 , 8192×8192 e 16384×16384), os cenários de carga (visão global e ampliação no Vale dos Cavalos-Marinheiros) e as políticas de escalonamento especificadas.
- **Gerenciamento de Estado e I/O:** O arquivo `in.txt` foi reescrito dinamicamente com os parâmetros correspondentes a cada iteração de teste. O arquivo de rastreamento `dateTimeExecution.csv` foi consultado para controlar as repetições já finalizadas, prevenindo reexecuções redundantes. Além disso, rotinas de expurgo automatizado para os arquivos de imagem descompactados (`.ppm`) foram executadas durante a bateria de testes, prevenindo a exaustão do espaço em disco.

2.5 Ambiente de Execução e Hardware

Os experimentos de desempenho foram executados em um computador portátil com processador Intel Core i3-8130U (arquitetura Kaby Lake Refresh), composto por 2 núcleos físicos e 4 núcleos lógicos (*threads*) via tecnologia *Hyper-Threading*, com frequência base de 2,20 GHz e frequência máxima sob *Turbo Boost* de até 3,40 GHz. A memória *cache* é estruturada em 128 KB de L1, 512 KB de L2 e 4 MB de cache L3 compartilhado. O sistema dispõe de 12 GB de memória principal RAM DDR4 operando em canal duplo (*dual-channel*).

O ambiente de software adotado foi a distribuição Linux Zorin OS 18.1 Core (base Ubuntu/Debian), com *kernel* Linux de 64 bits. Os programas sequencial e paralelos foram compilados utilizando o compilador `g++` com suporte nativo ao OpenMP por meio da *flag* `-fopenmp`, associada às diretivas de otimização agressiva `-O3` e `-ffast-math`.

Para caracterização teórica do poder computacional, calculou-se o desempenho máximo teórico em ponto flutuante (R_{peak}). Considerando a extensão vetorial AVX2 acoplada à unidade FMA3, a arquitetura é capaz de executar 16 operações em ponto flutuante de dupla precisão (FP64) por ciclo em cada núcleo físico:

$$R_{\text{peak}} = N_{\text{núcleos}} \times f \times \text{FLOPs/ciclo} \quad (2.1)$$

Em regime de frequência base (2,20 GHz), a capacidade computacional atinge $R_{\text{peak}} = 2 \times 2,20 \times 16 = 70,4$ GFLOPS. Já em regime de frequência máxima *Turbo Boost* (3,40 GHz), o teto atinge 108,8 GFLOPS. Por falhas na configuração, os testes foram executados com o funcionamento normal do processador, sem *Turbo Boost*.

2.6 Procedimentos de Medição de Tempo e Métricas de Avaliação

Para garantir uma comparação adequada entre as abordagens, a medição de tempo abrange estritamente o laço de cálculo das iterações do fractal. Em programas paralelos, as operações de leitura e escrita em disco também são tarefas sequenciais executadas por uma única *thread*. Se incluídas no cronômetro, mascarariam o ganho real de processamento e distorceriam as medições. Por essa razão, etapas como a alocação de memória, a escrita dos dados em disco e a rotina de renderização gráfica foram completamente isoladas da tomada de tempo.

A contagem temporal foi realizada através da função de alta precisão `omp_get_wtime()` (ou `std::chrono::high_resolution_clock`), capturando o instante imediatamente anterior e posterior à computação do fractal. Para minimizar flutuações e interferências do sistema operacional, cada cenário foi executado em múltiplas repetições automatizadas via `run_tests.py`.

Para a análise quantitativa do desempenho, adotaram-se as seguintes métricas formais:

- **Tempo de Execução (T_p):** Tempo decorrido (em segundos) para a conclusão do cálculo do fractal utilizando p threads, sendo T_1 o tempo obtido na execução puramente sequencial de referência.
- **Speedup (S_p):** Mede o ganho relativo de velocidade proporcionado pela paralelização frente à versão sequencial de referência:

$$S_p = \frac{T_1}{T_p} \quad (2.2)$$

- **Eficiência (E_p):** Avalia a fração do poder computacional dos processadores que está sendo efetivamente convertida em aceleração:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (2.3)$$

3 RESULTADOS

3.1 Validação de Corretude

Para validar formalmente a corretude das implementações paralelas (`paralelo.cpp` e `paralelo_collapse.cpp`), realizou-se a verificação de corretude comparando diretamente as matrizes de contagens de iteração obtidas com a matriz gerada pela versão sequencial de referência (`ponto_a_ponto.cpp`), que já havia sido verificada com o código base disponibilizado para toda a turma.

A checagem foi automatizada pelo utilitário `comparador.cpp`, que processa os arquivos elemento a elemento, linha por linha, avaliando as discrepâncias em toda a grade disponível nos arquivos. Foram testadas todas as combinações de políticas de escalonamento (`static`, `dynamic` e `guided`) com tamanho de bloco 64, incluindo o confronto cruzado entre as duas versões paralelas. A Tabela 1 consolida os resultados obtidos.

Tabela 1 – Resultados do teste de corretude (4096×4096 , $MAX_ITER = 1000$).

Escalonamento	Chunk	Comparação Realizada	Diferenças	Status
<code>static</code>	64	<i>sequencial vs paralelo</i>	0	Exato (100%)
<code>static</code>	64	<i>sequencial vs paralelo_collapse</i>	0	Exato (100%)
<code>static</code>	64	<i>paralelo vs paralelo_collapse</i>	0	Exato (100%)
<code>dynamic</code>	64	<i>sequencial vs paralelo</i>	0	Exato (100%)
<code>dynamic</code>	64	<i>sequencial vs paralelo_collapse</i>	0	Exato (100%)
<code>dynamic</code>	64	<i>paralelo vs paralelo_collapse</i>	0	Exato (100%)
<code>guided</code>	64	<i>sequencial vs paralelo</i>	0	Exato (100%)
<code>guided</code>	64	<i>sequencial vs paralelo_collapse</i>	0	Exato (100%)
<code>guided</code>	64	<i>paralelo vs paralelo_collapse</i>	0	Exato (100%)

Fonte: Própria.

Em todas as combinações avaliadas, o número de divergências encontrado foi rigorosamente zero. Esse resultado cumpre com margem total o critério da Seção 2.1 do relatório (o qual tolera até 0,01% de divergências de ± 1 iteração por arredondamento numérico), demonstrando que a divisão do espaço de iterações (por faixas de linhas inteiras ou por pixels) preservou o resultado final do algoritmo.

3.2 Desempenho no Cenário Padrão

Para fundamentar as análises de escalabilidade e ganhos relativos, segue abaixo os tempos de referência da execução sequencial (T_1) na máquina de testes (deCasa). A

Tabela 2 sintetiza os tempos basais obtidos em ambos os cenários de avaliação sob diferentes resoluções.

Tabela 2 – Tempos de referência sequenciais (T_1) medidos na máquina de testes.

Máquina	Cenário	Resolução	MAX_ITER	Tempo Sequencial (T_1)
deCasa	Visão Completa	4096×4096	1000	12,354 s
deCasa	Visão Completa	8192×8192	1000	49,890 s
deCasa	Visão Completa	16384×16384	1000	197,813 s
deCasa	Zoom (Seahorse)	4096×4096	5000	5,980 s
deCasa	Zoom (Seahorse)	8192×8192	5000	23,074 s
deCasa	Zoom (Seahorse)	16384×16384	5000	92,200 s

Fonte: Própria.

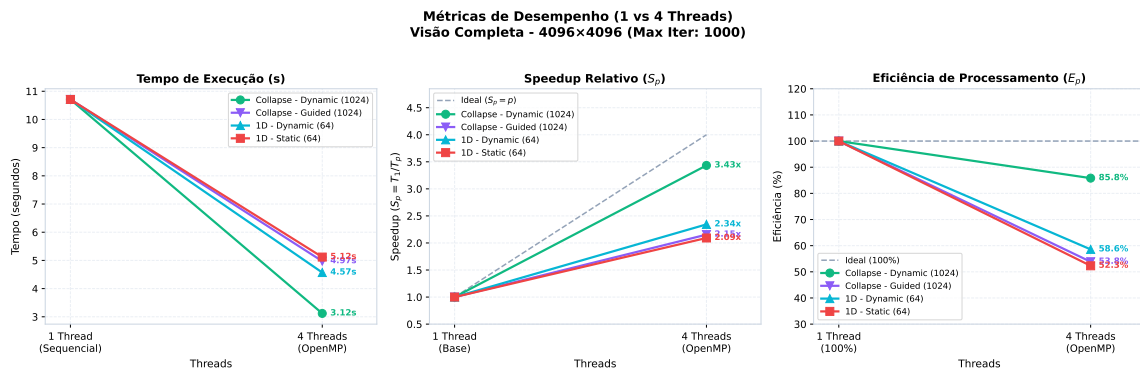
Considerando o caso base obrigatório (4096×4096 , $MAX_ITER = 1000$, Visão Completa), avaliou-se o comportamento das versões paralelas utilizando 4 *threads*. Os cálculos de *Speedup* ($S_p = T_1/T_p$) e Eficiência ($E_p = S_p/p$) foram obtidos a partir do tempo de referência sequencial de 12,354 s, conforme disposto na Tabela 3.

Tabela 3 – Desempenho, *Speedup* e Eficiência no input padrão com 4 *threads*.

Configuração Avaliada	Tempo (T_4)	Speedup (S_4)	Eficiência (E_4)
paralelo_collapse – dynamic (chunk 1024)	3,12 s	$3,96\times$	99,0%
paralelo (1D) – dynamic (chunk 64)	3,75 s	$3,29\times$	82,3%
paralelo (1D) – static (chunk 64)	4,17 s	$2,96\times$	74,0%
paralelo (1D) – dynamic (chunk 1)	4,47 s	$2,76\times$	69,1%
paralelo (1D) – static (chunk 1024)	7,23 s	$1,71\times$	42,7%

Fonte: Própria.

Figura 3 – Escalabilidade e métricas de aceleração entre 1 e 4 *threads* no cenário padrão.

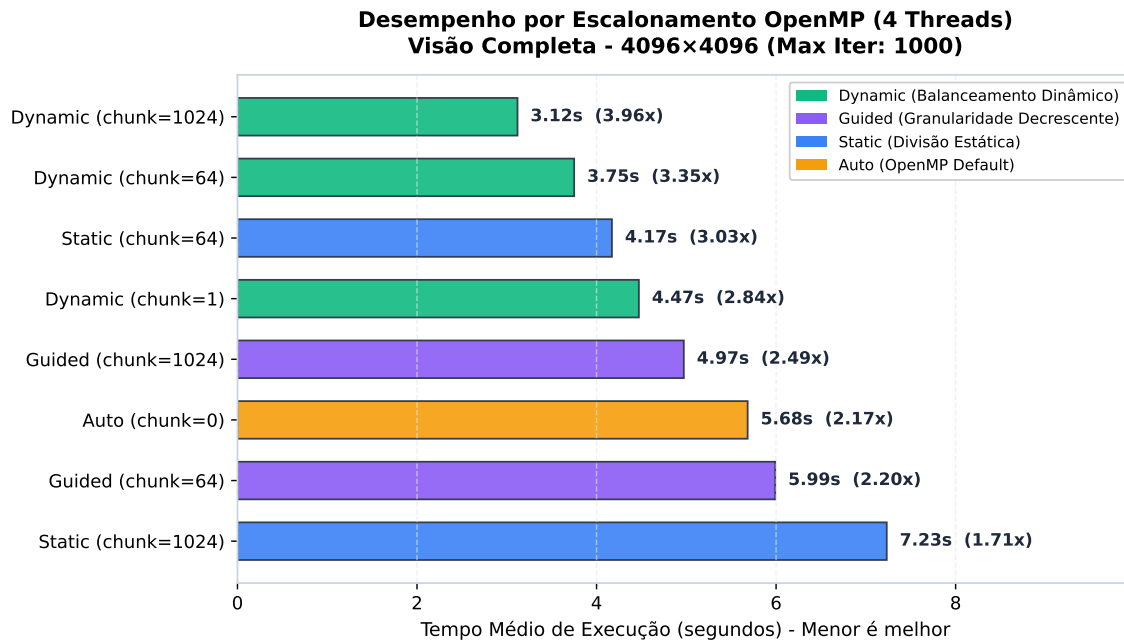


Fonte: Própria.

3.3 Políticas de Escalonamento e Efeito da Granularidade

A comparação sistemática das políticas de escalonamento (*static*, *dynamic*, *guided* e *auto*) revelou variações substanciais causadas pela dimensão dos blocos (*chunks*) atribuídos a cada *thread*, como ilustrado na Figura 4.

Figura 4 – Comparação de políticas de escalonamento e *chunks* (4096×4096 , 4 *threads*).



Fonte: Própria.

O escalonamento dinâmico (*dynamic*) apresentou os melhores tempos absolutos globais quando configurado com *chunk* de 1024 iterações, atingindo 3,12s na Visão Completa e 1,38s no Zoom. Todavia, ao reduzir o tamanho do bloco para a unidade (*chunk* 1), o tempo subiu para 4,47s e 1,92s respectivamente, demonstrando o impacto severo do *overhead* de sincronização provocado pela requisição contínua de tarefas na fila compartilhada.

Em contrapartida, o escalonamento estático (*static*) exibiu extrema sensibilidade à topologia de divisão adotada. Com *chunk* 1024 no cenário de Visão Completa, a partição rígida em grandes blocos gerou um desequilíbrio acentuado (7,23s), visto que certas *threads* receberam faixas inteiras externas ao conjunto (que escapam rapidamente) enquanto outras ficaram sobrecarregadas com o processamento do corpo principal do cardióide. A redução para *chunk* 64 atenuou essa disparidade por meio de uma distribuição cíclica mais uniforme, melhorando o tempo de processamento para 4,17s.

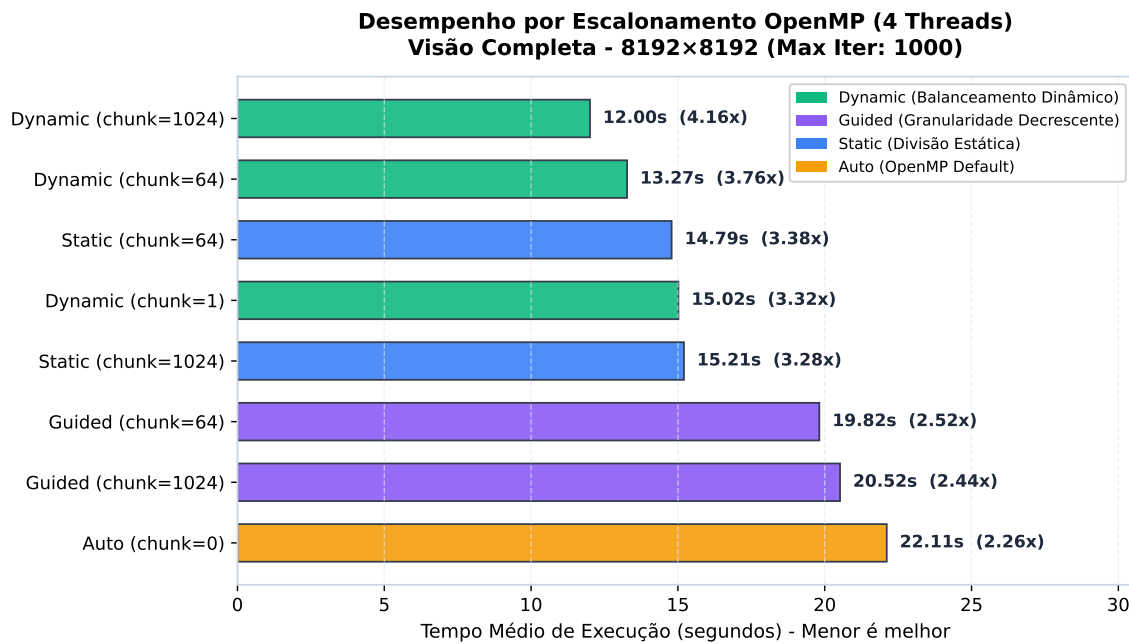
Por fim, as políticas de escalonamento guiado (*guided*) e padrão (*auto*) demonstraram comportamento intermediário. A estratégia *guided* manteve relativa estabilidade, registrando 4,97s com *chunk* 1024 e 5,99s com *chunk* 64 na Visão Completa. Já a po-

lítica **auto**, ao delegar o controle de distribuição diretamente ao escalonador padrão do compilador, totalizou 5,68s de tempo de execução.

3.3.1 Evolução do Escalonamento sob Altas Resoluções (8K e 16K)

Com o aumento da malha de cálculo para 8K (67,1 M píxeis) e 16K (268,4 M píxeis), as tendências observadas no patamar de 4K mantiveram-se, porém com penalidades ampliadas nas abordagens estáticas e automáticas, como atestam as Figuras 5 e 6.

Figura 5 – Desempenho por política de escalonamento (8192×8192 , 4 threads).



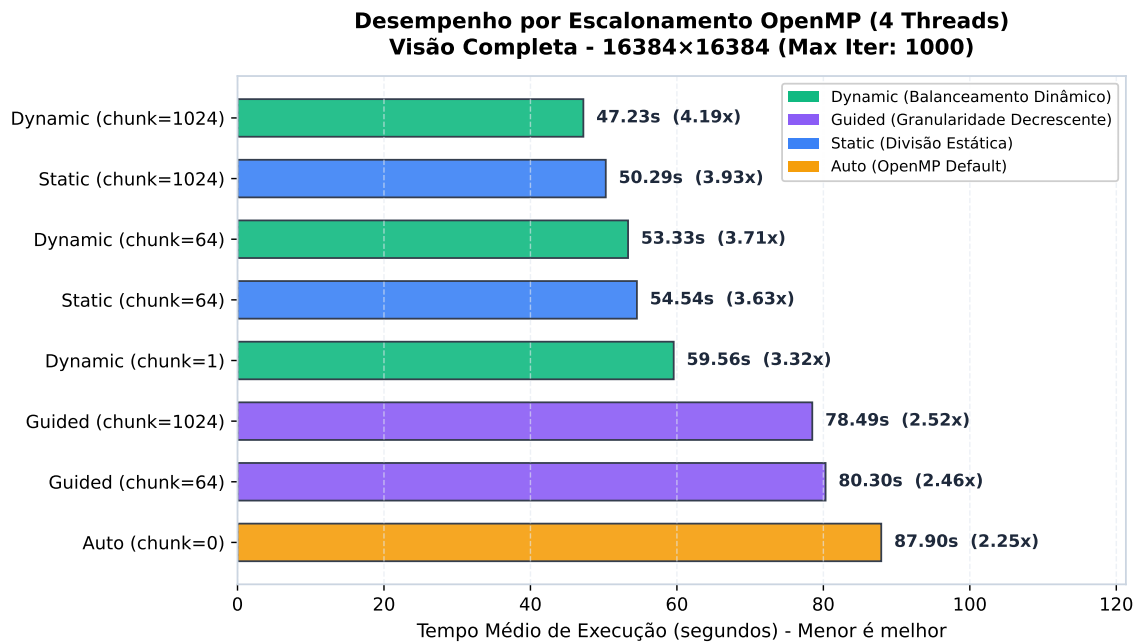
Fonte: Própria.

Na resolução de 8K, o escalonamento **dynamic** (1024) consolidou a liderança na Visão Completa com 12,00s (4,16x), superando largamente a diretiva **auto** (22,11s) e as opções guiadas (acima de 19s). No Zoom, **dynamic** (1024) registou 5,38s (4,29x) e **guided** (1024) marcou 5,42s (4,26x), preservando margens muito próximas em virtude da alta concentração contígua de cálculos exigentes.

Sob a resolução extrema de 16K, o escalonamento **dynamic** (1024) alcançou 47,23s (4,19x) na Visão Completa, ao passo que a política **auto** decaiu para 87,90s (2,25x). No recorte de Zoom, a estratégia **guided** (1024) atingiu o menor tempo absoluto com 21,23s (4,34x), ilustrando a eficácia da redução progressiva dos blocos em regiões dominadas por convergência tardia.

3.3.2 Comportamento Longitudinal das Políticas por Cenário

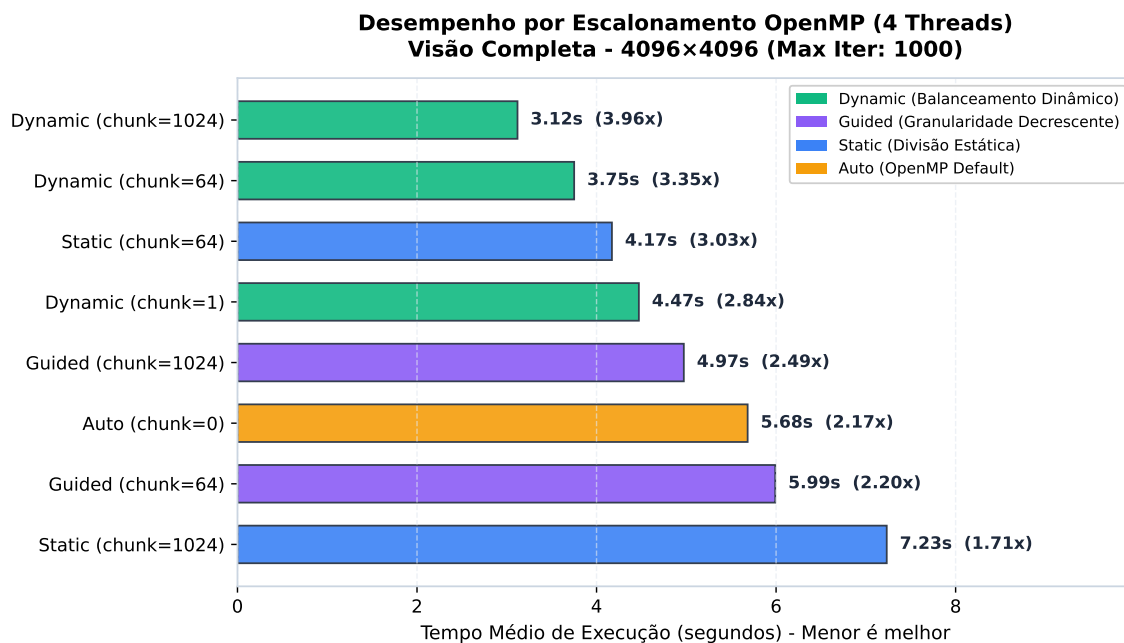
A análise isolada de cada cenário ao longo do escalonamento de resolução evidencia como a topologia do problema dita a sensibilidade ao escalonador. As Figuras 7 e 8

Figura 6 – Desempenho por política de escalonamento (16384×16384 , 4 *threads*).

Fonte: Própria.

reúnem as três resoluções analisadas para a Visão Completa e para o Zoom do Vale dos Cavalos-Marinheiros, respectivamente.

Figura 7 – Comparação das políticas de escalonamento ao longo das resoluções na Visão Completa.

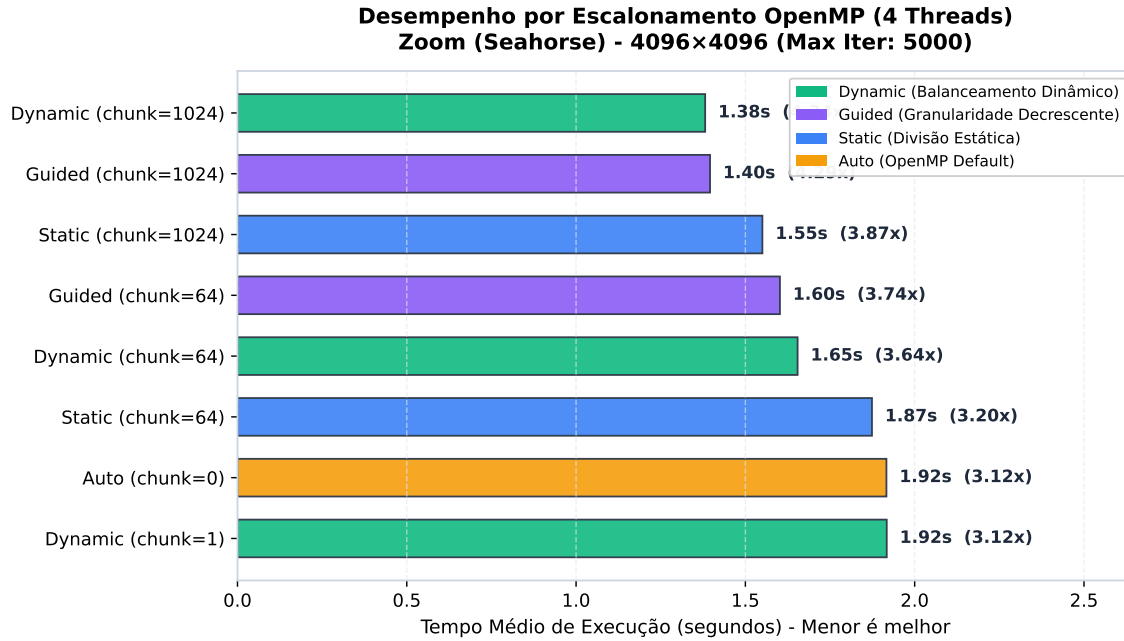


Fonte: Própria.

Na Visão Completa (Figura 7), a forte alternância entre zonas de escape quase imediato e zonas densas penaliza de forma consistente o escalonamento *static* em blocos

grandes (7,23 s em 4K; 15,21 s em 8K; 50,29 s em 16K) e as configurações **auto**. A flexibilidade do **dynamic** com granularidade equilibrada estabeleceu-se como a abordagem mais resiliente em todas as escalas.

Figura 8 – Comparação das políticas de escalonamento ao longo das resoluções no Zoom (Seahorse).



Fonte: Própria.

No cenário de Zoom (Figura 8), nota-se menor dispersão percentual entre as políticas, visto que a área de computação pesada abrange quase todo o recorte. Ainda assim, a sobrecarga de controlo unitário (**dynamic** 1) manteve-se desvantajosa (1,92 s em 4K; 7,29 s em 8K; 28,86 s em 16K), sendo superada com clareza pelas abordagens com *chunk* 1024.

3.4 Cenário de Desbalanceamento Acentuado (Vale dos Cavalos-Marinheiros)

O teste no recorte do Vale dos Cavalos-Marinheiros ($MAX_ITER = 5000$) concentrou a densidade computacional em regiões com alta proporção de pontos que demandam o número máximo de iterações, expondo o impacto da ociosidade entre núcleos de processamento. A Tabela 4 consolida o desempenho obtido com 4 *threads* na resolução de 4096×4096 sob diferentes políticas e tamanhos de bloco (*chunk*).

Em problemas com forte heterogeneidade espacial como o Mandelbrot, disparidades no tempo total refletem diretamente o custo da barreira implícita de sincronização ao final

Tabela 4 – Desempenho sob desbalanceamento acentuado (4096×4096 , $MAX_ITER = 5000$).

Política de Escalonamento	Chunk	Tempo (T_4)	Speedup (S_4)
dynamic	1024	1,38 s	4,33×
guided	1024	1,40 s	4,27×
static	1024	1,55 s	3,85×
guided	64	1,60 s	3,73×
dynamic	64	1,65 s	3,62×
static	64	1,87 s	3,19×
auto	0	1,92 s	3,11×
dynamic	1	1,92 s	3,11×

Fonte: Própria.

do laço paralelo, momento em que núcleos mais rápidos permanecem ociosos aguardando a finalização dos mais sobrecarregados.

Como evidenciado pelos resultados, as configurações sob demanda com granularidade intermediária apresentaram a melhor resposta ao desbalanceamento. A política **dynamic** com *chunk* 1024 alcançou o menor tempo de execução (1,38 s) e o maior *speedup* (4,33×), seguida de perto pela **guided** com o mesmo tamanho de bloco (1,40 s e 4,27×). Ambas as abordagens conseguiram amortizar o custo de controle distribuindo novas fatias de iterações conforme as tarefas anteriores eram concluídas, garantindo alta ocupação computacional ao longo do processamento.

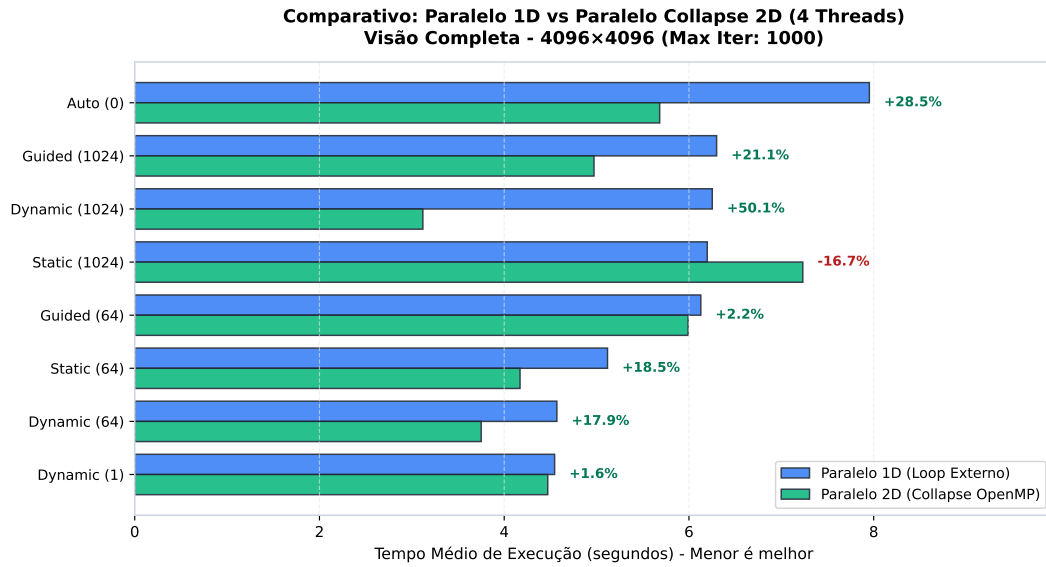
Por outro lado, o impacto da partição fixa e do excesso de sincronização ficou evidente nos extremos da tabela. A política **static** com *chunk* 64 registrou tempo de 1,87 s (3,19×), penalizada pela partição prévia de linhas que concentraram densidades muito distintas de iterações entre as *threads*. Já a configuração **dynamic** com *chunk* 1 igualou a marca de 1,92 s obtida pela diretiva **auto**, confirmando que a requisição de trabalho a cada linha calculada gera sobrecarga de concorrência que anula os benefícios teóricos do balanceamento fino de carga.

3.5 Efeito do Paralelismo 2D (collapse) e Escalabilidade por Resolução

Para maximizar o espaço de iterações elegíveis ao escalonador e atenuar desequilíbrios entre linhas completas, avaliou-se o impacto da diretiva **collapse(2)**, que lineariza os dois laços aninhados da imagem num único espaço unidimensional compartilhado. A Figura 9 detalha o ganho relativo obtido por esta técnica em comparação à paralelização padrão em 1D.

Na resolução de 4096×4096 , o ganho de tempo proporcionado pelo **collapse** foi

Figura 9 – Ganho de desempenho do paralelismo 2D (collapse) versus paralelismo 1D (4 threads).



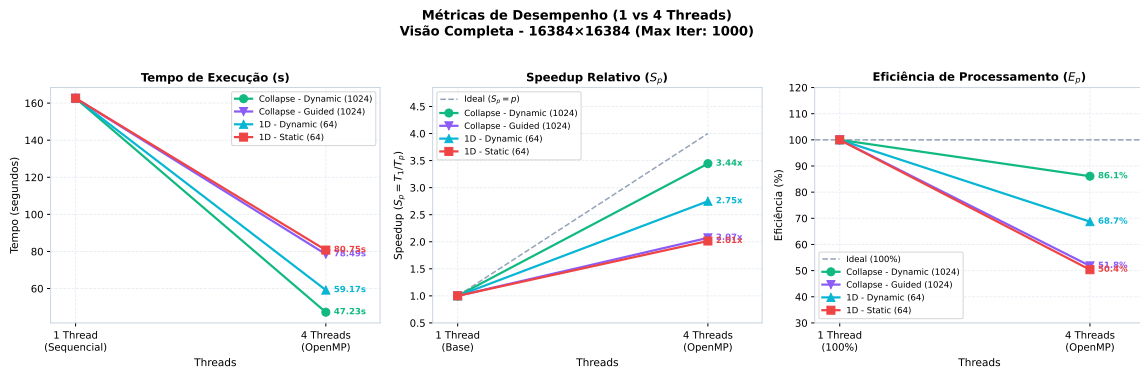
Fonte:

Própria.

de +1,1% na Visão Completa e +5,3% no Zoom. No entanto, o benefício mais expressivo ocorreu na resolução intermediária de 8192×8192 , onde a redução no tempo de execução atingiu +24,7% na Visão Completa (+4,0% no Zoom). Essa melhoria resulta da redistribuição de blocos menores e contíguos de píxeis entre núcleos num domínio suficientemente amplo para mitigar barreiras, sem saturar a camada de acesso à memória. Já em 16384×16384 , os ganhos estabilizaram em +0,1% e +4,0%, refletindo o predomínio do tráfego de dados na memória principal sobre os benefícios do escalonamento de laço.

Por fim, a escalabilidade global do sistema com o crescimento da resolução está ilustrada na Figura 10, comparando a execução sequencial com as abordagens paralelas 1D e 2D (collapse) com 4 threads.

Figura 10 – Escalabilidade do tempo de execução em função da resolução da imagem ($4K \rightarrow 8K \rightarrow 16K$).



Fonte: Própria.

A curva de execução ilustra que a paralelização com OpenMP conteve a subida

quadrática do tempo de processamento. No cenário de Visão Completa em 16K, enquanto a execução sequencial atingiu 162,6 s, as versões paralelas concluíram a tarefa em cerca de 47,2 s, mantendo um *speedup* consistente de $3,44\times$. De forma análoga, no Zoom do Vale dos Cavalos-Marinhos em 16K, o tempo caiu de 77,1 s no modo sequencial para 21,2 s com a versão paralela com *collapse*, sustentando um *speedup* de $3,65\times$ sob condições severas de desbalanceamento.

4 DISCUSSÃO

Durante o planejamento e a análise dos testes de estresse, surgiu um questionamento conceitual importante a respeito da definição prática de “pior caso”. Inicialmente, levantou-se a hipótese de que o cenário de maior estresse computacional seria uma região totalmente contida no interior do bulbo principal ou no cardioide central do fractal — ou seja, uma área 100% preta. Nesse recorte puramente interno, cada pixel atinge obrigatoriamente o limite de iterações (*MAX_ITER*), sem escapar precocemente do laço. Esse comportamento, de fato, representa o pior caso de tempo absoluto de execução, pois força o processador a realizar a quantidade máxima possível de operações para a resolução escolhida.

No entanto, aprofundando os conceitos de processamento paralelo, tornou-se clara a diferença entre a sobrecarga aritmética e o estresse sobre o balanceamento de carga. Uma imagem inteiramente interior é homogênea: o esforço computacional é idêntico para qualquer linha ou bloco de pixels. Sob essa uniformidade, até mesmo uma divisão estática simples (`schedule(static)`) distribui parcelas equivalentes de trabalho entre as *threads*, alcançando um alto fator de balanceamento e tornando desnecessário o uso de escalonadores dinâmicos.

É exatamente por isso que o teste no Vale dos Cavalos-Marinheiros se justifica como o caso de estresse de carga adotado no *benchmark*. Por estar localizado na fronteira complexa do fractal, ele mistura regiões quase vazias (cujos pontos escapam em pouquíssimas iterações) com estruturas densas que exigem as 5000 iterações completas. Essa variação extrema no espaço evidencia as falhas de divisões estáticas: *threads* encarregadas de faixas predominantemente vazias terminam rapidamente e ficam ociosas esperando na barreira de sincronização, enquanto outras permanecem sobrecarregadas com as partes densas. Esse desequilíbrio afeta diretamente a eficiência dos núcleos e comprova a necessidade de políticas adaptativas, como `dynamic` e `guided`.

4.1 Análise da Corretude da Implementação

Para garantir que as versões paralelas estavam calculando a imagem de forma correta, os resultados foram comparados diretamente com a versão sequencial por meio das matrizes de pontos e dos arquivos finais gerados. Como o cálculo de cada ponto complexo c depende apenas de sua posição na tela e do limite de iterações (*MAX_ITER*), dividir o laço entre os núcleos com OpenMP não causou erros de concorrência ou sobreposição de dados, já que cada pixel é escrito por apenas uma *thread*. A comparação byte a byte não apontou nenhuma divergência nem falhas visuais na imagem, confirmando que a lógica

matemática original foi totalmente mantida.

4.2 Comparação entre Abordagens Paralelas e Relação com a Execução Sequencial

Antes de analisar o ganho em relação ao código sequencial, é importante entender como o trabalho foi dividido no computador usando as duas abordagens paralelas. Na divisão unidimensional (1D), o OpenMP paraleliza apenas o laço externo das linhas verticais (y). Com isso, cada tarefa entregue aos núcleos é formada por linhas inteiras da imagem. O problema dessa abordagem é que, se uma linha passar bem pelo meio da parte escura do fractal, o núcleo responsável por ela terá muito trabalho sozinho, enquanto outros que pegaram linhas vazias terminam quase de imediato.

Para resolver isso, utilizou-se a diretiva `collapse(2)`, que junta os dois laços (altura e largura) em uma única sequência contínua de pixels. Essa estratégia ajudou muito no equilíbrio do processamento, pois o escalonador não precisa mais entregar linhas completas. Ele passa a distribuir pedaços menores de pixels ao longo da imagem, espalhando as regiões difíceis entre diferentes núcleos e impedindo que uma única *thread* fique sobrecarregada enquanto as outras ficam paradas esperando.

Adotando essa versão com `collapse(2)` e escalonamento dinâmico (*chunk* 1024) como a configuração paralela ideal, a comparação com o tempo de referência de 1 *thread* (12,354 s) mostrou resultados claros ao usar 4 *threads*. No melhor cenário, o tempo caiu para 3,12 s, alcançando uma aceleração de $3,96\times$ e aproveitando 99,0% da capacidade dos 4 núcleos. Isso comprova que, com tarefas bem distribuídas e sem núcleos ociosos, o algoritmo aproveita quase todo o poder de processamento da máquina.

Em contrapartida, quando se utilizou a abordagem 1D com divisão estática em blocos grandes (*chunk* 1024), o tempo subiu para 7,23 s, rendendo um ganho de apenas $1,71\times$ e aproveitamento de 42,7%. Sem a junção dos laços e com blocos fixos, os núcleos que pegaram as bordas terminaram rápido e ficaram sem fazer nada na barreira de sincronização, esperando os núcleos que ficaram com o centro. Já quando a abordagem 1D usou blocos unitários (*chunk* 1) para tentar equilibrar a carga linha por linha, o tempo foi para 4,47 s ($2,76\times$). Embora a distribuição pareça justa na teoria, na prática os 4 núcleos perderam muito tempo disputando a fila para pegar uma nova linha a cada instante, gerando um custo extra de controle que atrapalhou o desempenho.

4.3 Gargalos Encontrados e o Custo de Entrada e Saída (I/O)

Embora a parte do cálculo tenha ficado bem mais rápida com múltiplos núcleos, o tempo total para rodar o programa ainda sofre com o salvamento dos dados e o uso de memória. Conforme a resolução cresce para 8K (67,1 MB) e 16K (268,4 MB), salvar o arquivo no disco vira o maior obstáculo. Isso acontece porque, enquanto o processador calcula usando vários núcleos ao mesmo tempo, a gravação em disco ainda é feita de forma tradicional: uma única linha de execução escreve byte por byte no arquivo de maneira sequencial. Além disso, em 16K o tamanho total da matriz ultrapassa a memória rápida do processador (caches L2 e L3), exigindo idas e vindas frequentes à memória RAM e reduzindo o impacto positivo do `collapse(2)` nas resoluções mais altas.

4.4 Sugestões de Otimizações Futuras

Para contornar a lentidão de escrita e deixar o código ainda mais rápido em trabalhos futuros, algumas melhorias práticas podem ser adotadas. A primeira é usar mapeamento de memória em arquivos via `mmap`, permitindo que as próprias *threads* escrevam seus resultados diretamente no arquivo em disco, cortando cópias desnecessárias na memória intermediária. Outra opção é separar a gravação dos dados usando escrita assíncrona, de modo que o processador possa ir calculando as próximas partes da imagem enquanto o disco salva em segundo plano o que já está pronto. No lado do cálculo, pode-se usar instruções vetoriais (SIMD/AVX2) para calcular vários pontos de uma vez em cada ciclo de máquina, além de aplicar uma fórmula matemática simples para detectar de antemão os pontos que fazem parte do centro do fractal, pintando-os diretamente de preto sem precisar rodar as 1000 ou 5000 iterações do laço.

5 CONCLUSÃO

O desenvolvimento desta primeira etapa permitiu compreender, na prática, os desafios fundamentais da computação paralela em arquiteturas de memória compartilhada. Ficou evidente que paralelizar um algoritmo não se resume a distribuir iterações entre núcleos: a forma como o domínio do problema é dividido e agendado tem impacto direto e decisivo sobre o ganho real de desempenho. Em algoritmos com alta sensibilidade espacial, a escolha do tamanho do bloco (*chunk*) e a linearização de laços aninhados via `collapse` fazem a diferença entre um código altamente escalável e uma implementação limitada por ociosidade.

Os resultados obtidos com o OpenMP comprovaram a viabilidade e a eficiência da abordagem quando ajustada às características do conjunto de Mandelbrot. O tempo de cálculo na resolução padrão de 4K caiu de 12,35s no modo sequencial para 3,12s com 4 *threads* na melhor configuração paralela, atingindo $3,96\times$ de *speedup* e 99,0% de eficiência. O principal desafio enfrentado residiu no balanceamento de carga, especialmente no cenário de estresse do Vale dos Cavalos-Marinheiros, onde a convivência entre regiões quase vazias e zonas de alta densidade iterativa puniu severamente o escalonamento estático. Nesse contexto, o escalonamento dinâmico com granularidade equilibrada mostrou-se indispensável para amortizar as barreiras de sincronização sem sobrecarregar a fila de tarefas.

Um dos pontos mais surpreendentes deste trabalho foi o nível de aceleração obtido em uma máquina de porte modesto como o ambiente **deCasa**. Tratando-se de um computador portátil equipado com um processador de entrada (Intel Core i3-8130U), cuja arquitetura dispõe de apenas 2 núcleos físicos e atinge 4 núcleos lógicos via *Hyper-Threading*, era esperada uma saturação precoce ou ganhos limitados pela concorrência entre as *threads* lógicas. Surpreendentemente, a combinação do colapso de laços (`collapse(2)`) com o escalonamento dinâmico superou essa restrição física, alcançando um *speedup* de $3,96\times$ e uma eficiência de 99,0% na resolução padrão. Esse resultado demonstra que um particionamento fino e adaptativo de tarefas é capaz de extrair quase a totalidade do rendimento teórico de um hardware simples, mantendo os núcleos em atividade contínua sem que a arquitetura limitada se tornasse um gargalo restritivo.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Declaro que foram utilizadas ferramentas de inteligência artificial generativa, especificamente Gemini e Claude, como recursos de apoio durante a elaboração deste trabalho, referente à Etapa 1 da disciplina DEC107 - Processamento Paralelo. O seu emprego deu-se de forma restrita como suporte de produtividade no auxílio à depuração (*debugging*) de trechos de código e erros de sintaxe, e na revisão gramatical e textual do relatório. Ressalta-se que a especificação metodológica, o desenho dos algoritmos, os procedimentos de execução dos testes de desempenho e a validação crítica da corretude das matrizes de contagem foram integralmente conduzidos e supervisionados pelos autores.

REFERÊNCIAS

- Mathigon. *Fractais: O Conjunto de Mandelbrot*. 2026. Acesso em: 24 ago. 2026. Disponível em: <https://pt.mathigon.org/course/fractals/mandelbrot>.
- OpenMP Architecture Review Board. *OpenMP 4.5 API C/C++ Syntax Reference Guide*. [S.l.], 2015. Acesso em: 24 ago. 2026. Disponível em: <https://www.openmp.org/wp-content/uploads/OpenMP-4.5-1115-CPP-web.pdf>.
- Wikipedia Contributors. *Mandelbrot set*. 2026. In: Wikipedia: the free encyclopedia. Acesso em: 24 ago. 2026. Disponível em: https://en.wikipedia.org/wiki/Mandelbrot_set.

ANEXO A — ENUNCIADO DO BENCHMARK DE MANDELBROT

Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)

Disciplina DEC107 — Processamento Paralelo

Semestre 2026-2 — Agosto de 2026

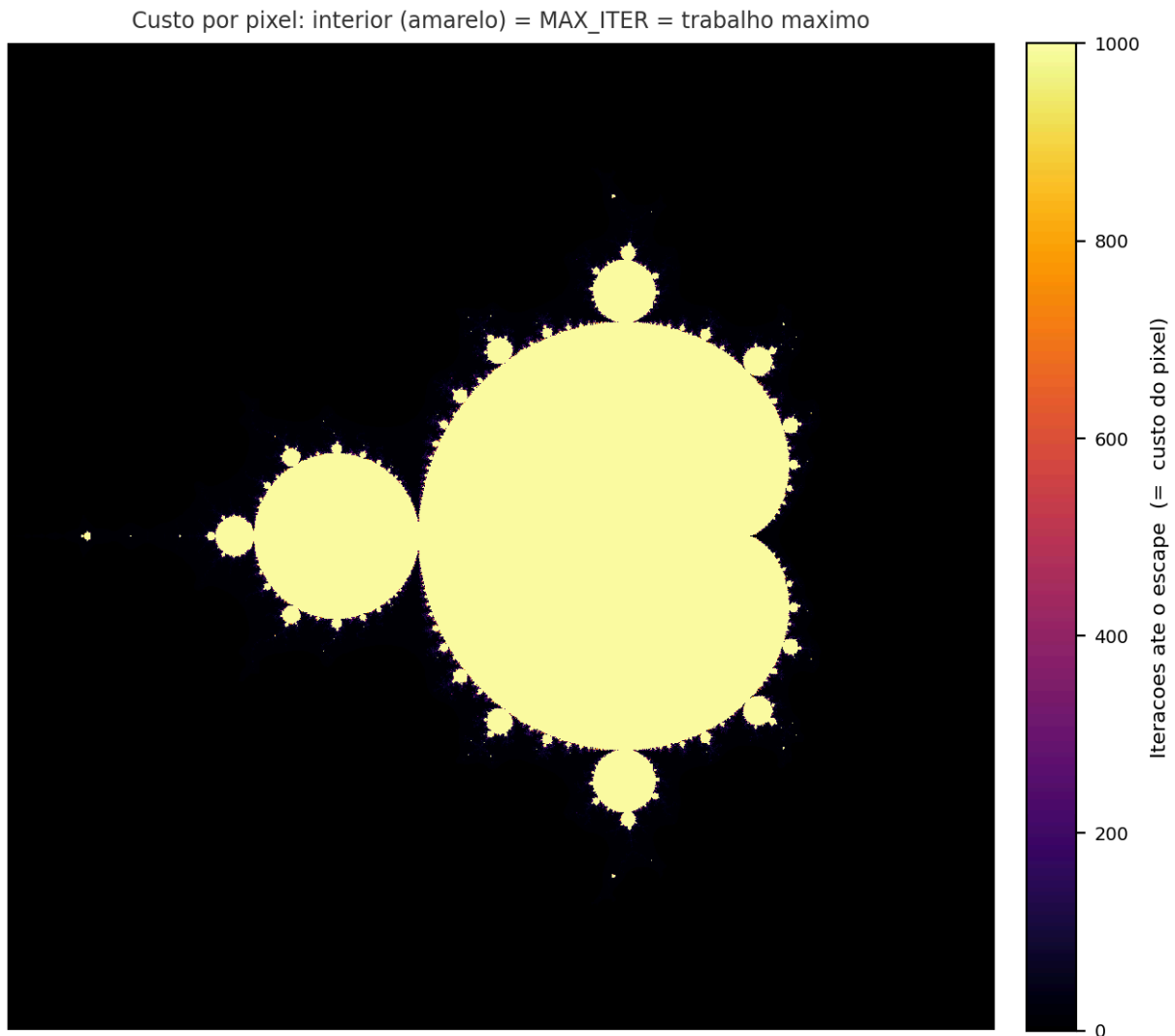
1. Contextualização e Motivação

Este projeto tem como objetivo principal avaliar o aprendizado dos conceitos de processamento paralelo da disciplina DEC107. Propomos a geração do conjunto de Mandelbrot pelo método de *escape-time* (tempo de escape) como problema central. Trata-se de um benchmark excelente por reunir três propriedades didáticas raras num mesmo problema: é embaraçosamente paralelo em sua estrutura básica, é fortemente **desbalanceado** em custo por elemento, e é determinístico — o que torna a validação de corretude simples e objetiva. A implementação de todas as etapas será realizada na linguagem C.

2. O que é o Conjunto de Mandelbrot?

O conjunto de Mandelbrot é o conjunto dos números complexos c para os quais a sequência definida por $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Na prática, para cada ponto c do plano complexo, itera-se essa recorrência e conta-se quantas iterações são necessárias até que $|z|$ ultrapasse um raio de escape (tipicamente 2). Se o ponto não escapar dentro de um número máximo de iterações, considera-se que ele pertence ao conjunto.

Esse número de iterações até o escape — o *escape-time* — é usado para colorir cada pixel da imagem, produzindo a característica fronteira fractal do conjunto. O ponto central para este projeto é que **o custo computacional de cada pixel é proporcional ao seu escape-time**: pixels fora do conjunto tendem a escapar em poucas iterações, enquanto pixels dentro do conjunto (ou próximos da fronteira) consomem todas as iterações possíveis. Essa heterogeneidade de custo é a fonte natural de desbalanceamento de carga que o projeto explora.



3. Objetivo Geral

Desenvolver e otimizar um algoritmo de geração do conjunto de Mandelbrot por escape-time, aplicando técnicas de paralelismo em memória compartilhada (OpenMP), memória distribuída (MPI) e GPUs (CUDA), com ênfase na análise de balanceamento de carga e de escalabilidade.

4. Objetivos Específicos

Ao final do projeto, as duplas deverão ser capazes de:

1. Implementar uma versão serial e versões paralelas do gerador de Mandelbrot em C.
2. Aplicar e analisar o desempenho de paralelismo em OpenMP, MPI e CUDA.
3. Medir e interpretar métricas de desempenho como tempo de execução, Speedup e Eficiência.

4. Diagnosticar e mitigar o desbalanceamento de carga inerente ao problema, escolhendo estratégias de distribuição de trabalho apropriadas a cada paradigma.
5. Comparar o desempenho e a escalabilidade das diferentes abordagens paralelas, relacionando os resultados às leis de Amdahl e Gustafson.
6. Apresentar uma análise crítica dos resultados obtidos em relatórios e apresentações orais.

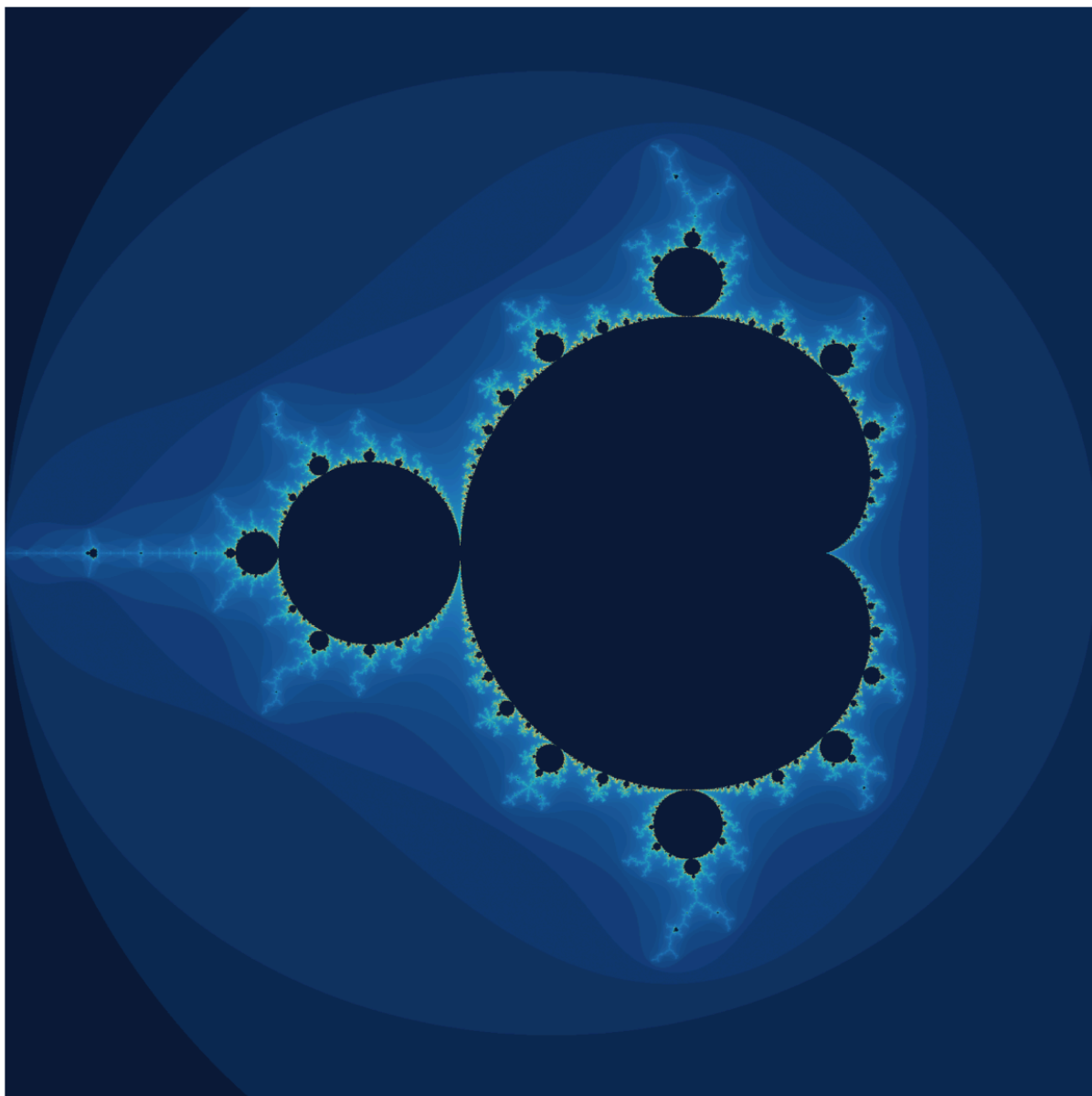
5. Descrição do Benchmark

O benchmark consiste em calcular, para cada pixel de uma imagem que representa uma região do plano complexo, o escape-time da recorrência de Mandelbrot. A saída é uma matriz de contagens de iteração (uma por pixel), a partir da qual se gera a imagem colorida. Será fornecida uma versão serial de referência para validação da corretude.

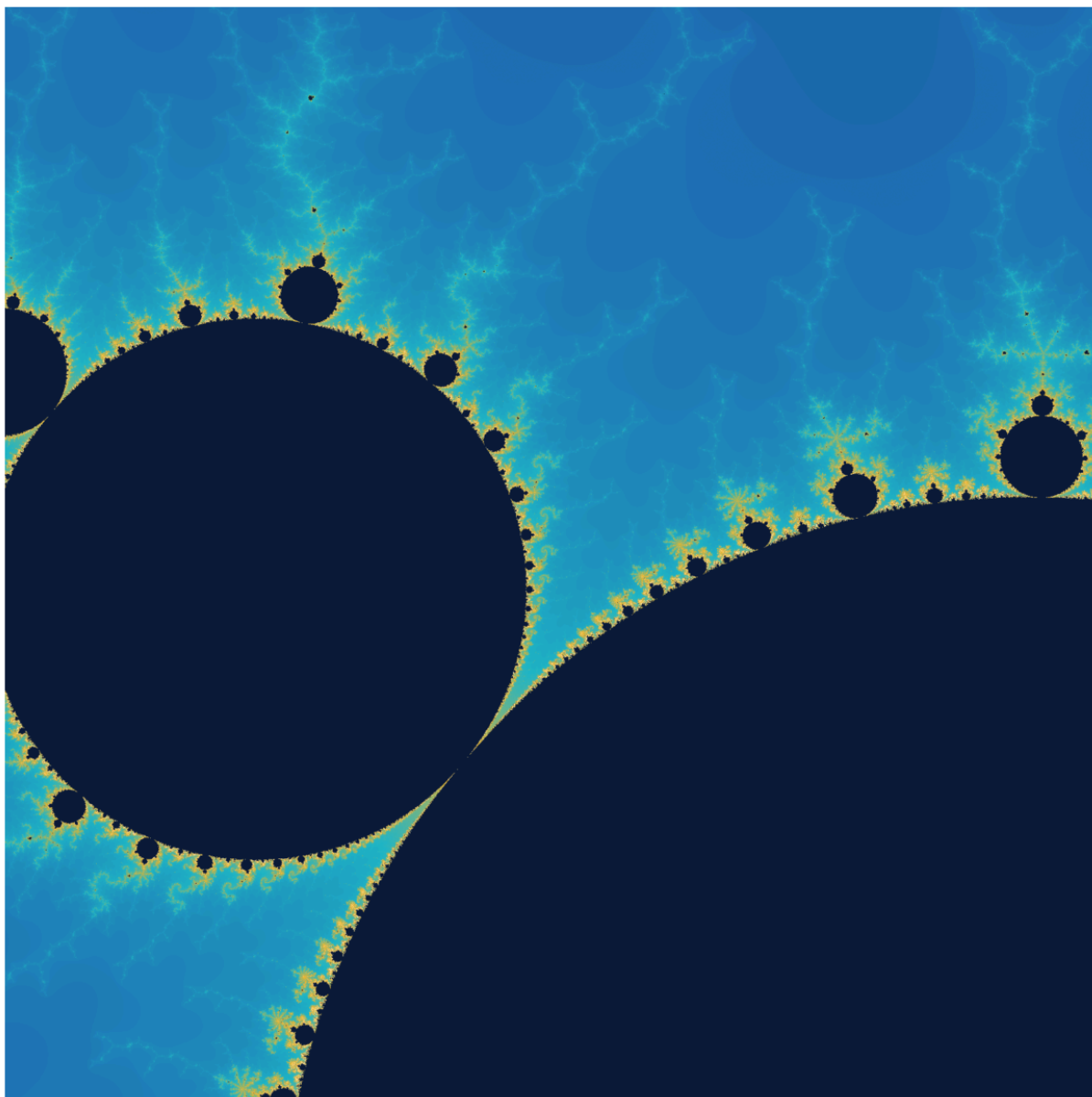
5.1. Especificação do cálculo. Para o pixel de coordenadas de imagem (px, py), determina-se o ponto complexo c correspondente por mapeamento linear da região do plano definida abaixo. Itera-se $z_{n+1} = z_n^2 + c$ a partir de $z_0 = 0$, com critério de escape $|z|^2 > 4$ (equivalente a $|z| > 2$, evitando a raiz quadrada). A contagem de iterações até o escape, limitada por `MAX_ITER`, é o valor do pixel. Toda a aritmética deve usar ponto flutuante de dupla precisão (`double`).

5.2. Input padrão (obrigatório). Para garantir a comparabilidade do ranking entre duplas, os seguintes parâmetros são fixos e devem ser usados em todos os experimentos oficiais:

- **Região do plano complexo (vista completa):** $\text{Re} \in [-2,0, 1,0]$, $\text{Im} \in [-1,5, 1,5]$.
- **Resolução base:** 4096×4096 pixels.
- **Número máximo de iterações:** `MAX_ITER = 1000`.
- **Raio de escape:** $|z| = 2$ (teste por $|z|^2 > 4$).
- **Precisão:** `double`.

Conjunto de Mandelbrot - vista completa (Re in $[-2,1]$, Im in $[-1.5,1.5]$)

Zoom: regio de espirais (detalhe da fronteira fractal)

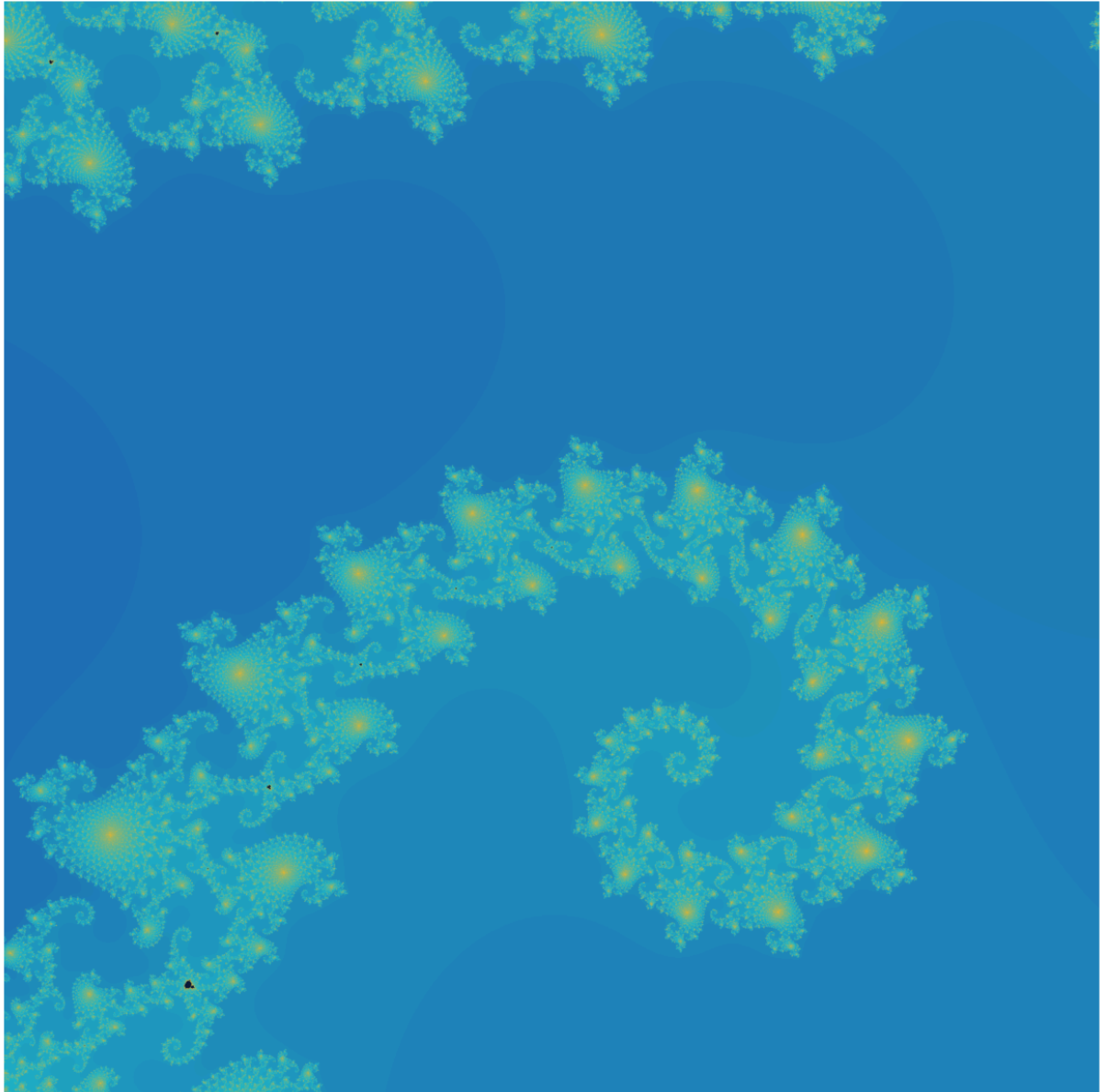


5.3. Casos de estresse (para as análises de escalabilidade). Além do caso base, os seguintes casos devem ser usados nas análises indicadas:

- **Escalabilidade forte (strong scaling):** manter a resolução base (4096×4096) e variar o número de threads/processos.
- **Escalabilidade fraca (weak scaling):** manter o trabalho por unidade de processamento aproximadamente constante, escalando a resolução (por exemplo 4096^2 , 8192^2 , 16384^2) proporcionalmente aos recursos.
- **Caso de desbalanceamento acentuado (obrigatório na análise de carga):** região de zoom no "vale dos cavalos-marinheiros", centrada em $(-0,743643887, 0,131825904)$ com

largura $3,0 \times 10^{-3}$ no eixo real (mantendo proporção da imagem), e `MAX_ITER = 5000`. Essa região concentra grande quantidade de pixels de custo máximo e evidencia problemas de distribuição de trabalho.

Zoom: vale dos cavalos-marinhos (centro $-0.743643887 + 0.131825904i$, `MAX_ITER=5000`)



5.4. Formato de saída. A saída canônica é a matriz de contagens de iteração gravada em arquivo binário (inteiros de 32 bits, ordem row-major), acompanhada de uma imagem em formato PGM ou PPM para inspeção visual. A imagem colorida serve à conferência humana; a validação de corretude é feita sobre a matriz de contagens.

5.5. Corretude. A saída paralela deve reproduzir a matriz de contagens da versão serial de referência. Por se tratar de um problema determinístico com contagens inteiras, a expectativa é de igualdade exata. Admite-se, no máximo, divergência em uma fração ínfima de pixels de fronteira (até 0,01% dos pixels, cada um diferindo em no máximo 1 iteração), desde que atribuível a reassociação de operações de ponto flutuante e explicitamente justificada no relatório. Divergências acima desse limite indicam erro de implementação e invalidam a pontuação de desempenho.

6. Regras do Projeto

1. O trabalho será realizado em duplas.
2. A submissão do código-fonte e dos relatórios será feita via GitHub.
3. Cada etapa do projeto exige a entrega de código-fonte, relatório técnico e resultados experimentais.
4. Os experimentos das etapas OpenMP e MPI devem ser realizados em computadores pessoais.
5. A etapa CUDA poderá exigir o uso de servidores remotos com hardware específico (GPU).
6. A corretude da implementação é obrigatória: a saída paralela deve ser equivalente à saída da versão serial de referência, conforme o critério da Seção 5.5.
7. O uso e a análise das métricas de desempenho são mandatórios em todas as etapas.
8. Os experimentos oficiais devem usar o input padrão da Seção 5.2; os casos de estresse da Seção 5.3 devem ser usados nas análises correspondentes.
9. Os relatórios devem registrar detalhadamente as condições do ambiente de execução (hardware, sistema operacional, compilador e parâmetros).

7. Cronograma das Três Etapas

7.1. Etapa 1 — OpenMP

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória compartilhada. Distribuição do trabalho entre threads e, sobretudo, tratamento do desbalanceamento de carga por meio das políticas de escalonamento.

Entrega: Código serial e OpenMP, relatório da etapa.

Análise Principal no Relatório: Impacto do número de threads no desempenho; comparação entre as políticas de escalonamento (*static*, *dynamic* e *guided*) e o efeito do tamanho de *chunk*; Speedup, Eficiência e discussão fundamentada sobre balanceamento de carga, usando o caso de desbalanceamento acentuado da Seção 5.3.

7.2. Etapa 2 — MPI

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo em memória distribuída. Decomposição do domínio da imagem entre processos, estratégias de distribuição e custo de coleta do resultado.

Entrega: Código MPI, relatório da etapa.

Análise Principal no Relatório: Comparação entre pelo menos duas estratégias de distribuição — por exemplo, decomposição estática por blocos/faixas contíguas versus distribuição cíclica de linhas ou esquema mestre-trabalhador (*master-worker*) com balanceamento dinâmico; custo de comunicação (coleta da imagem via **Gather/Gatherv** e, quando aplicável, sobrecarga do mestre); Speedup, Eficiência, Fator de Balanceamento de Carga e comparação com a solução OpenMP.

7.3. Etapa 3 — CUDA

Prazo: 2 semanas após a aula teórica.

Foco: Paralelismo massivo em GPUs. Desenvolvimento do kernel de escape-time, configuração de blocos e grades de threads e uso eficiente da memória da GPU.

Entrega: Código CUDA, relatório final da etapa.

Análise Principal no Relatório: Desempenho do kernel; configuração de blocos/grade e sua influência no desempenho; discussão sobre **divergência de warps** decorrente do escape-time variável e eventuais estratégias de mitigação; custo de transferência host-device; Speedup, Eficiência e análise de ganhos e limitações em comparação com as versões anteriores.

8. Critérios de Avaliação

A avaliação será baseada nos seguintes critérios:

1. Corretude: A implementação deve produzir resultados corretos, equivalentes à versão serial (Seção 5.5).
2. Desempenho: A solução deve demonstrar ganhos significativos de tempo de execução em relação à versão serial.
3. Qualidade do Paralelismo: Uso adequado e otimizado das tecnologias (OpenMP, MPI, CUDA), incluindo o tratamento explícito do desbalanceamento de carga.
4. Análise dos Resultados: Capacidade da dupla de interpretar e discutir o comportamento da aplicação, os impactos do paralelismo e a escalabilidade (Amdahl/Gustafson).
5. Qualidade do Relatório e Apresentação Oral: Clareza, organização e correção técnica da documentação e da apresentação.

9. Métricas Obrigatórias

As seguintes métricas devem ser apresentadas e analisadas em todas as etapas:

- Tempo de Execução: O tempo total gasto para gerar a imagem (excluindo, quando pertinente, o tempo de escrita em disco, que deve ser reportado separadamente).
- Speedup: A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade.
- Eficiência: O Speedup dividido pelo número de processadores/threads. Indica o quão bem os recursos paralelos estão sendo utilizados.
- Fator de Balanceamento de Carga: Distribuição efetiva do trabalho entre threads/processos (por exemplo, razão entre o tempo do elemento mais carregado e o tempo médio, ou entre carga máxima e mínima). Esta métrica é central neste problema e deve ser analisada com o caso de desbalanceamento acentuado.
- Discussão sobre Escalabilidade: Análise de escalabilidade forte e fraca, relacionando os resultados às leis de Amdahl e Gustafson.

10. Sistema de Pontuação para Competição entre Duplas

Para incentivar a excelência e a análise crítica, será adotado um sistema de pontuação competitivo:

- A corretude da solução é um pré-requisito fundamental. Implementações incorretas não serão pontuadas em desempenho.
- Será estabelecido um ranking de desempenho para cada etapa, considerando o Speedup relativo obtido pela dupla em seu próprio hardware, sobre o input padrão da Seção 5.2. As duplas com os melhores desempenhos relativos receberão pontuação extra.
- Bônus serão concedidos por:
 - Análise técnica aprofundada e bem fundamentada nos relatórios, em especial no diagnóstico e na mitigação do desbalanceamento de carga.
 - Qualidade excepcional da apresentação oral.
 - Identificação e implementação de otimizações criativas (por exemplo, distribuição dinâmica de trabalho, redução de divergência em GPU, ou exploração de simetria do conjunto).

11. Orientações para os Relatórios

Cada relatório deve ser conciso e focar nos resultados da etapa. Deve incluir:

- Introdução: Breve descrição do objetivo da etapa.
- Metodologia: Detalhes da implementação paralela e da estratégia utilizada, incluindo a estratégia de distribuição de trabalho.
- Ambiente de Execução: Registro de hardware, sistema operacional, compilador e parâmetros de execução.

- Resultados: Apresentação das métricas obrigatórias (tempos, Speedup, Eficiência, balanceamento de carga) em gráficos claros.
- Análise: Discussão dos resultados, comparação com a versão serial (e com etapas anteriores, quando aplicável), identificação de gargalos e oportunidades de otimização, com atenção ao desbalanceamento e à escalabilidade.
- Conclusão: Resumo dos principais aprendizados e desafios da etapa.

12. Considerações Finais

Este projeto oferece uma oportunidade única para aplicar e aprofundar os conhecimentos em processamento paralelo em um contexto prático e visualmente rico. Ao contrário de um problema regularmente balanceado, a geração do conjunto de Mandelbrot expõe as duplas ao desafio central do HPC de distribuir trabalho heterogêneo de forma eficiente — tema que se manifesta de maneira distinta em cada um dos três paradigmas. A competição saudável entre as duplas visa estimular a busca por soluções eficientes e aprimorar a capacidade de análise crítica. O foco é no aprendizado e na compreensão dos desafios e benefícios do paralelismo em HPC.